LLM Judge Bias and Sensitivity

Inference Pipeline — Architecture Document

ETH Zürich — Training Better Preference Models

Swiss AI Stack · Multi-Model Support

March 2026

Abstract

This document describes the architecture of a research pipeline for measuring systematic biases and sensitivities in LLM-as-a-judge evaluations. The system is a pure HTTP client that dispatches judge queries to the Swiss AI Stack, an OpenAI-compatible inference endpoint hosted at CSCS. We detail the data pipeline, the five core experiments (position bias, template sensitivity, reasoning depth, input sensitivity, and user-prompt structure), and an OPRO-based prompt optimisation module that automatically discovers system prompts with reduced positional bias.

Contents

1 Overall Architecture

The pipeline is a **pure HTTP client** that sends judge queries to the Swiss AI Stack at https://api.swissai.cscs.ch/v1. There is no local GPU cluster, no Slurm scheduler, and no local vLLM instance. All inference is performed remotely via OpenAI-compatible REST requests; the local code handles data loading, prompt construction, asynchronous dispatch, response parsing, metric computation, and prompt optimisation.

Key terminology. Every API call sends exactly one *system prompt* and one *user prompt*. The system prompt is a hidden instruction that defines the judge's persona and output format (e.g. "You are an expert rater…"). The user prompt contains the evaluation data: the original user question and two candidate responses. The system prompt varies across experiments B1–B4; both the system prompt *and* the user prompt structure vary in experiment B5 (Section 3.5).

1.1 Data-Flow Overview

Figure 1 illustrates the high-level data flow. The dataset module downloads and caches preference data locally. At experiment time, pairs are loaded from disk, combined with prompt templates, and dispatched as asynchronous HTTP requests. Responses are parsed, saved as JSONL, and fed to the metrics module for analysis.

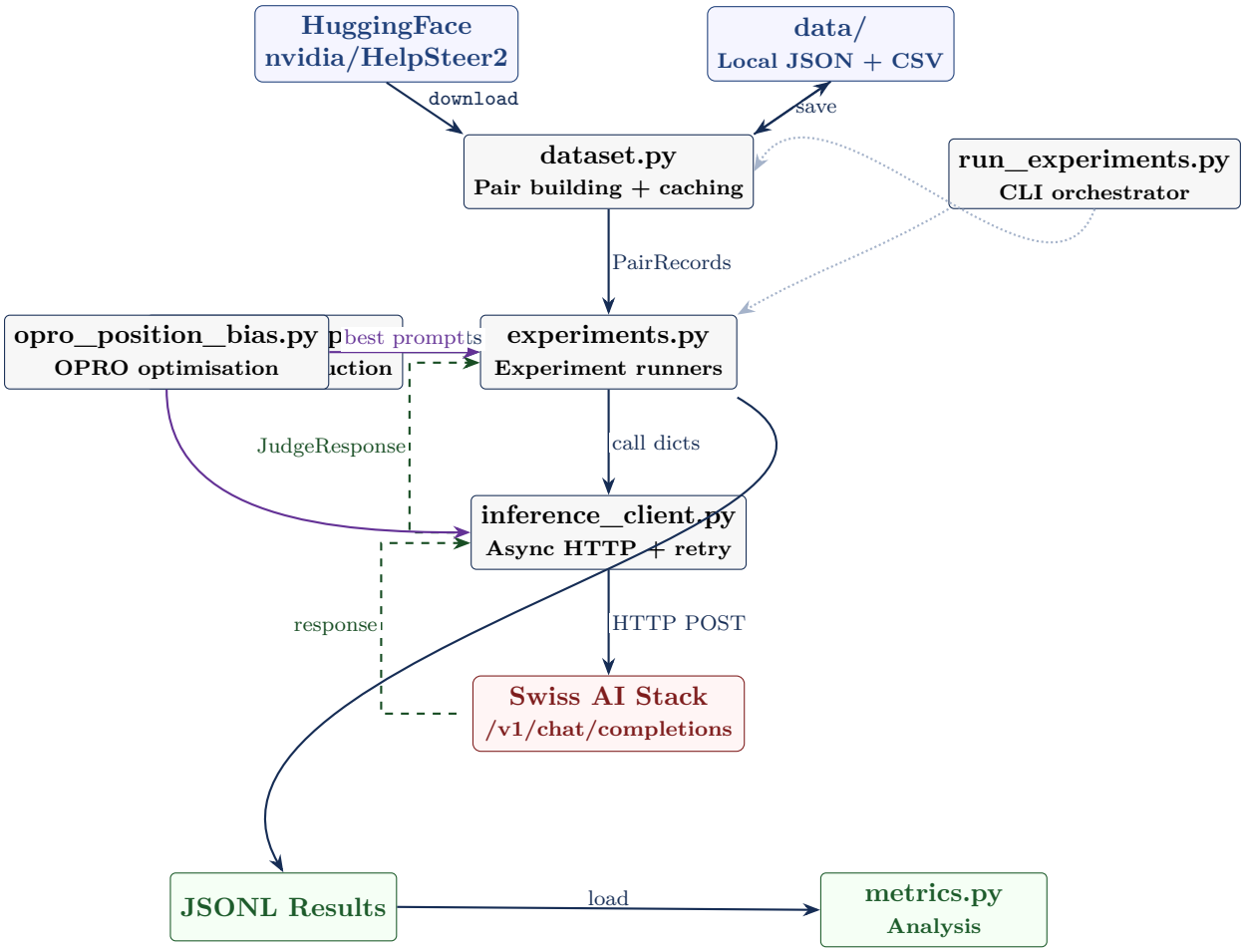

Figure 1: High-level data flow of the evaluation pipeline. Solid arrows indicate the forward path; dashed arrows show the return path. Purple arrows represent the OPRO prompt optimisation loop.

2 Module Descriptions

2.1 `dataset.py` — Data Pipeline

This module handles all interactions with the HelpSteer2 dataset. The pipeline operates in two phases:

Download phase. The `download_dataset()` function fetches HelpSteer2 from HuggingFace, groups responses by prompt, and constructs *all pairwise combinations* within each group. For each pair, a composite quality score is computed as

$$s = \frac{\text{helpfulness} + \text{correctness} + \text{coherence}}{3}.$$

The higher-scoring response is placed as A (gold label = "A"); pairs with equal scores receive gold label "C". Results are saved locally as both JSON and CSV in the `data/` directory.

The standard file now includes five *stratification fields* alongside every pair (Table 1). The full file additionally includes per-response attribute scores, composite scores, and response lengths.

Table 1: Stratification fields written by `download_dataset()`.

Field	Type	Description
`score_gap`	float	$\lvert s_A - s_B \rvert$; absolute quality difference
`difficulty`	str	"easy" (> 1.0), "medium" (> 0.33), "hard" (≤ 0.33)
`verbosity_delta`	int	$\text{verbosity}_A - \text{verbosity}_B$ (signed); positive = better response is more verbose
`complexity_max`	int	$\max(\text{complexity}_A, \text{complexity}_B)$; 0–4 scale
`is_multiturn`	bool	True if the prompt contains a prior `Assistant:` turn

Load phase. At experiment time, `load_dataset_pairs()` reads from local JSON, applies a seeded shuffle, and returns up to n `PairRecord` objects. No network access is required at this stage. Two optional sampling modes are available:

- **`stratify=True`**: sample proportionally across difficulty strata (easy / medium / hard) so that hard and medium pairs are not under-represented. A purely random sample is dominated by easy pairs (large quality gap), which are the stratum where positional bias is least pronounced.
- **`randomize_position=True`**: randomly flip 50% of pairs so that `gold_label` is approximately 50% A / 50% B. Without this, the best response is always placed as A, making any A-preference in the judge indistinguishable from accuracy. This flag is required for valid positional bias measurement.

The `PairRecord` dataclass carries the five base fields (`prompt_id`, `prompt`, `response_a`, `response_b`, `gold_label`) plus the five stratification fields from Table 1 as optional attributes (default `None` for legacy files). The `flipped()` method returns a copy with A and B swapped, the gold label inverted (A $\leftrightarrow$ B, C unchanged), and `verbosity_delta` negated (the other metadata fields are order-symmetric).

2.2 `inference_client.py` — Async HTTP Client

This module wraps the Swiss AI Stack's OpenAI-compatible endpoint with structured request/response handling, concurrency control, and retry logic.

Configuration. The `InferenceConfig` dataclass holds the endpoint URL, API key, model identifier, generation parameters (`max_tokens`=2048, `temperature`=0.0), and retry settings. The model is configurable; `check_models.py` queries the `/v1/models` endpoint to list currently available models.

Label extraction. After receiving a response, the module extracts the judge verdict through a three-tier cascade:

1. Explicit verdict lines (e.g. "Verdict: B", "Answer: 1").
2. A bare A/B/C or 1/2 at the end of the text (end-anchored pattern).
3. Fallback: the last standalone A/B/C anywhere in the text.

Labels `1` and `2` (produced by blind-framing templates in B5) are normalised to A and B respectively at extraction time, keeping all downstream metric functions compatible. The digit patterns are applied only in end-anchored positions to avoid false positives from numbers in response text. If the model produced a `<think>...</think>` block, it is stripped before extraction and stored separately.

Concurrency and retries. The `SwissAIClient` uses an `asyncio.Semaphore` (default 8) to bound the number of concurrent HTTP requests. On rate-limit responses (HTTP 429) or connection errors, it retries with exponential backoff: delays of 2, 4, 8, 16, 32 seconds, capped at 60 seconds, for up to 5 attempts.

2.3 `templates.py` — Prompt Templates

All system prompts are defined here and registered in a `TEMPLATES` dictionary. Table 2 lists the available templates.

Table 2: System prompt templates used across experiments.

Template ID	Framing	Experiment
`expert_rater`	Baseline expert rater	B1, B2, B3, B4, B5
`llm_judge`	"LLM used to judge"	B2
`neutral`	Imperative, no persona	B2
`academic`	Formal quality evaluator	B2
`minimal`	One-liner	B2
`reason_then_judge`	Explain reasoning, then verdict	B3
`structured_reasoning`	Rate on criteria, then verdict	B3
`expert_rater_alt1-3`	Minor wording variants	B4
`opro`	OPRO-optimised (low position bias)	B1
User-prompt structure variants (B5)		
`blind`	Responses labelled 1/2; criterion after	B5
`criterion_first`	A/B labels; criterion before responses	B5
`blind_criterion_first`	1/2 labels; criterion before responses	B5

Five evaluation criteria are supported: `helpful`, `quality`, `accurate`, `harmless`, and `concise`, each mapping to a natural-language phrase. The `build_prompt()` function resolves a template ID and criterion into a (system prompt, user prompt) pair.

For experiments B1–B4 the user prompt follows a fixed format:

```
[User Prompt]
{prompt}
```

```
[Response A]
{response_a}

[Response B]
{response_b}

Which response is {criterion}?
```

Experiment B5 introduces three structural variants of this user prompt, varying two binary dimensions independently (see Section 3.5).

2.4 `experiments.py` — Experiment Runners

Four asynchronous functions implement the core experiments. Each builds a list of call dictionaries, dispatches them via `batch_judge()`, and saves results to JSONL. Table 3 summarises the experimental conditions.

Table 3: Experiment functions and their conditions.

Function	ID	Conditions
`run_position_bias`	B1	AB (original), BA (flipped)
`run_template_sensitivity`	B2	5 templates
`run_reasoning_depth`	B3	3 reasoning conditions
`run_input_sensitivity`	B4	$4 \times 5 = 20$ (templates $\times$ criteria)
`run_user_prompt_structure`	B5	$4 \times 2 = 8$ (structures $\times$ orders)

2.5 `metrics.py` — Metric Computation

This module loads JSONL results and computes experiment-specific metrics:

- `compute_position_bias`: consistency rate, bias direction, and positional preference breakdown.
- `compute_pairwise_agreement`: overall and per-pair agreement across conditions (used for B2 and B4).
- `compute_thinking_accuracy`: accuracy versus gold labels, agreement with the no-reasoning baseline, and average latency per condition (used for B3).
- `compute_stratified_metrics`: apply any of the above functions separately for each value of a stratification field (e.g. `difficulty`), returning a dict keyed by stratum value. Enables per-stratum comparison of position consistency, template sensitivity, and accuracy.

2.6 CLI Orchestrators

`run_experiments.py` is the main entry point. It loads the dataset, configures the inference client, runs selected experiments, computes metrics, and prints summaries. Key flags include `-experiments`, `-n-pairs`, `-template` (including `opro`), `-model`, `-concurrency`, `-stratify`, and `-randomize-position`. When `-stratify` is active and difficulty labels are present, B1 results are automatically broken down by stratum using `compute_stratified_metrics`.

`run_opro.py` runs the OPRO optimisation loop (Section 3.6). It loads training and validation pairs, invokes the optimiser, and prints the best prompt with its scores.

`check_models.py` queries the `/v1/models` endpoint and lists all currently available models on the Swiss AI Stack—useful since model availability changes frequently.

3 Experiments

3.1 B1: Position Bias

Research question. Does the LLM judge prefer whichever response appears in a particular position, regardless of content quality?

Method. For each pair, the judge is queried twice: once with the original ordering (condition AB) and once with responses swapped (condition BA). A consistent judge should prefer the same underlying response in both orderings—if the AB verdict is "A", the BA verdict should be "B".

Metrics.

- *Position consistency*: fraction of pairs where the flipped verdict matches the expected inversion.
- *Bias toward first/second position*: fraction where the model always selects the response in position A (or B), regardless of content.
- *Tie inconsistency*: cases where one ordering produces a tie and the other does not.

3.2 B2: Template Sensitivity

Research question. How sensitive is the judge's verdict to the framing of the system prompt, given identical data and response ordering?

Method. Each pair is judged with five system prompt templates (Table 2), keeping data, order, and criterion constant. Only the judge persona changes.

Metrics. Overall pairwise agreement across templates, label distribution per template, and identification of the most volatile pairs (lowest cross-template agreement).

3.3 B3: Reasoning Depth

Research question. Does explicitly prompting the model to reason before giving its verdict improve accuracy against human gold labels?

Method. Three conditions are compared, differing only in the system prompt:

1. *No reasoning*: output only A, B, or C.
2. *Reason then judge*: explain strengths and weaknesses, then give a verdict.
3. *Structured reasoning*: rate each response on helpfulness, accuracy, and coherence, then give a verdict.

All reasoning is prompt-elicited—the prompt itself requests the reasoning. No native API thinking-budget feature is used.

Metrics. Accuracy versus gold labels per condition, delta accuracy over the no-reasoning baseline, and average latency (reasoning conditions produce longer outputs).

3.4 B4: Input Sensitivity

Research question. How sensitive is the judge to minor paraphrasing of the system prompt and changes in the evaluation criterion?

Method. Four semantically equivalent variants of the `expert_rater` template are crossed with five evaluation criteria, yielding $4 \times 5 = 20$ conditions per pair.

Metrics. Pairwise agreement across conditions and criterion drift (how much the label changes when only the criterion changes, with the template held fixed).

3.5 B5: User-Prompt Structure

Research question. Do structural properties of the *user prompt* affect judge verdicts, independently of system-prompt wording? Specifically: (1) do alphabetically loaded response labels (A/B) introduce bias compared to neutral labels (1/2), and (2) does placing the evaluation criterion *before* the responses improve accuracy by priming the judge?

Method. Two binary factors are crossed in a 2×2 design, holding the system prompt body constant across all conditions (only the output instruction changes to match the label type):

Table 4: B5 experimental conditions.

Condition	Labels	Criterion position
`expert_rater` (baseline)	A/B	After responses
`blind`	1/2	After responses
`criterion_first`	A/B	Before responses
`blind_criterion_first`	1/2	Before responses

Each condition is evaluated in both AB and BA response orderings (8 calls per pair total), enabling per-condition position consistency to be computed alongside cross-condition agreement.

Metrics.

- *Per-condition position consistency*: reuses the B1 metric to measure whether blind labels or criterion placement reduce positional instability.
- *Cross-condition pairwise agreement*: measures how much the label changes when the user prompt structure changes (AB order only).
- *Accuracy versus gold labels*: whether criterion-first placement improves alignment with human judgments.

Pilot results ($n = 10$, criterion = helpful). Table 5 reports preliminary results on a 10-pair smoke test.

Table 5: B5 pilot results ($n = 10$, `helpful` criterion, validation split). All figures are proportions.

Condition	Pos. consistency	Bias 1st	Bias 2nd	Acc. vs. gold
`expert_rater`	0.90	0.10	0.00	0.50
`blind`	0.80	0.00	0.20	0.40
`criterion_first`	0.60	0.20	0.20	0.50
`blind_criterion_first`	0.70	0.00	0.30	0.40
Cross-condition agreement		0.817		

Observations. The pilot reveals several noteworthy patterns (to be confirmed at scale):

- Switching from A/B to 1/2 labels *reverses the direction of positional bias*: the first-position preference seen with A/B labels (70% of AB verdicts are "A") disappears entirely, replaced

by a second-position preference (60% "B"), suggesting a learned A-letter preference in the model's training distribution.
- Criterion-first placement *increases* position sensitivity rather than reducing it (consistency drops to 0.60), contrary to the initial hypothesis. Pre-framing may cause the judge to anchor on the criterion too early and become inconsistent when responses are reordered.
- Cross-condition agreement of 0.817 indicates that roughly 18% of pairs receive different labels depending on user prompt structure alone—a non-negligible source of variance even when the underlying system prompt and data are identical.

These findings are preliminary ($n = 10$); a full run at $n = 200$ is required before drawing conclusions.

3.6 OPRO: Prompt Optimisation for Position Bias

Goal. Automatically discover a system prompt that minimises position bias, using the LLM itself to iteratively generate and refine candidates.

Method. The optimisation follows the OPRO framework (Optimization by PROmpting):

1. Evaluate two seed prompts (`expert_rater` and `llm_judge`) on a fixed subset of training pairs, measuring position consistency.
2. For N iterations: present the LLM with the full history of (prompt $\rightarrow$ score) pairs and ask it to generate a new system prompt. Evaluate the candidate on the same training subset.
3. Select the prompt with the highest training score.
4. Validate on a held-out set to assess generalisation.

A fixed subset of training pairs (default 80) is used for all evaluations, ensuring that scores are comparable across iterations. The best prompt is saved to `results/opro/opro_results.json` and registered as the `opro` template.

Transferability. OPRO-generated prompts are model-specific. A prompt optimised for one model (e.g. Qwen) may not generalise to another (e.g. Llama), as different architectures respond differently to debiasing instructions. When switching models, the optimisation should be re-run.

OPRO usage

```
# Default: 10 iterations, 80 eval pairs, 150 validation pairs
python run_opro.py

# Use the optimised prompt in experiments
python run_experiments.py --template opro --experiments position_bias
```

4 Concurrency and Retry Logic

The pipeline uses Python's `asyncio` event loop with `aiohttp` for non-blocking HTTP. All experiment calls are dispatched simultaneously via `asyncio.gather()`, bounded by an `asyncio.Semaphore`.

Concurrency control. The semaphore (default 8) ensures that at most 8 HTTP requests are in flight at any time, preventing server overload while maintaining high throughput.

Exponential backoff. On HTTP 429 (rate limit) or connection errors, the client waits $\min(\text{base\_delay} \times 2^{\text{attempt}}, 60)$ seconds before retrying. With a base delay of 2 s, the sequence is 2, 4, 8, 16, 32 s (capped at 60 s). After 5 failed attempts, a `RuntimeError` is raised.

Figure 2 illustrates the retry flow.

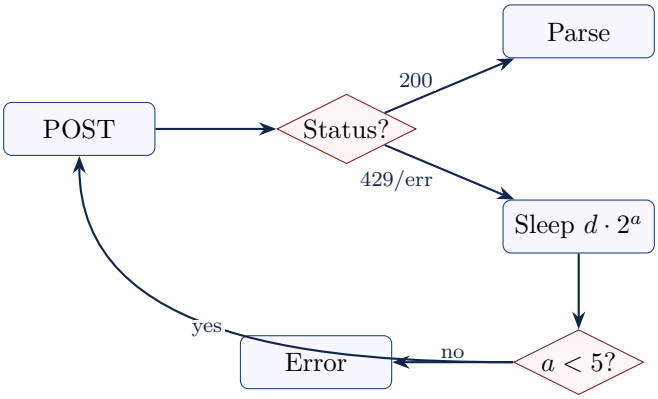

Figure 2: Exponential backoff retry logic. On rate-limit or error responses, the client retries up to 5 times with increasing delays.

5 Output Format

Each experiment writes a JSONL file to the `results/` directory, with one JSON object per judge call. Table 6 describes the schema.

Table 6: Fields in each JSONL result record.

Field	Type	Description
`prompt_id`	str	Unique identifier for the dataset pair
`experiment_id`	str	Experiment name
`condition`	str	Experimental condition (e.g. AB, BA, template name)
`raw_text`	str	Full model output, unmodified
`label`	str | null	Extracted verdict: A, B, C, or null
`thinking`	str | null	Extracted reasoning block, if present
`latency_s`	float	Request latency in seconds
`total_tokens`	int	Total tokens (prompt + completion)
`parse_ok`	bool	Whether label extraction succeeded
`error`	str | null	Error message, if the request failed

The results directory is organised as follows:

```
results/
  position_bias/
    expert_rater_helpful.jsonl
    opro_helpful.jsonl
  template_sensitivity/
    criterion_helpful.jsonl
  reasoning_depth/
    criterion_helpful.jsonl
  input_sensitivity/
    all.jsonl
  user_prompt_structure/
    criterion_helpful.jsonl
  opro/
    opro_results.json
```

6 Infrastructure

This pipeline performs no local GPU inference. All model calls are HTTP requests to the Swiss AI Stack. Table 7 summarises the infrastructure.

Table 7: Infrastructure summary.

Component	Detail
Inference backend	Swiss AI Stack (CSCS)
Endpoint	`https://api.swissai.cscs.ch/v1`
API compatibility	OpenAI `/v1/chat/completions`
Model	Configurable (use `check_models.py`)
Authentication	Bearer token via `SWISSAI_API_KEY`
Client library	`aiohttp` (async)
Concurrency	Semaphore-bounded (default 8)
Retry strategy	Exponential backoff, max 5 retries
Local requirements	Python 3.10+, aiohttp, datasets, numpy
Local GPU / cluster	None

7 Dataset Analysis

The following figures are generated by `dataset_analysis.ipynb` and provide an overview of the HelpSteer2 dataset used in all experiments.

7.1 Rating Distributions and Correlations

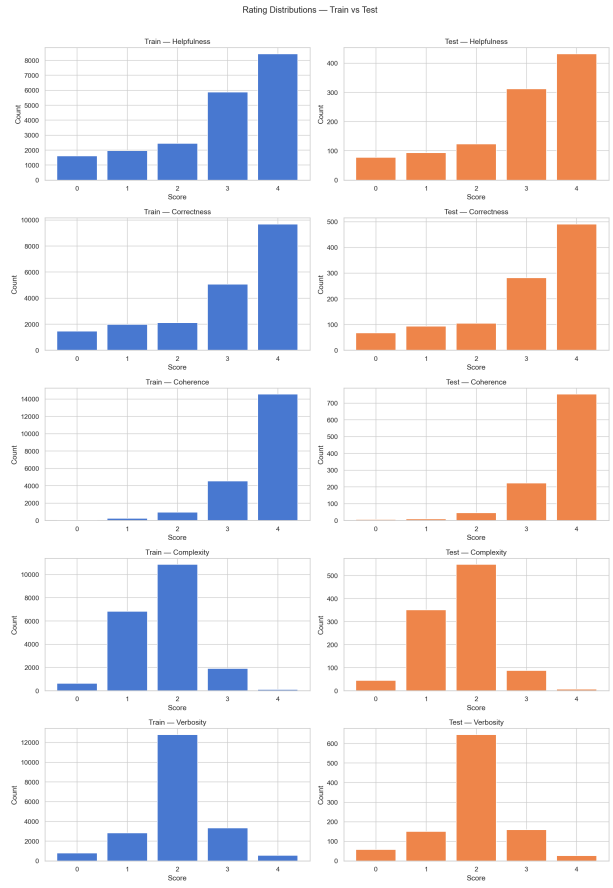

Figure 3: Distribution of helpfulness, correctness, coherence, complexity, and verbosity ratings across all responses.

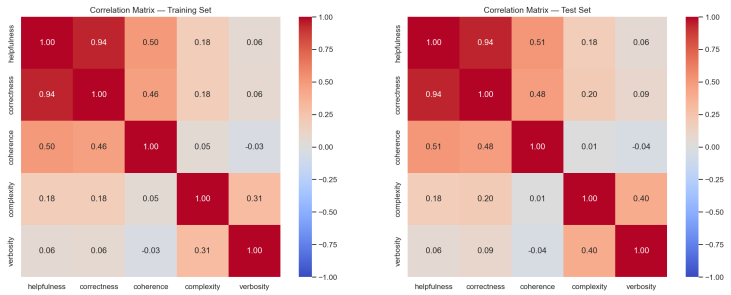

Figure 4: Pairwise Pearson correlation between rating dimensions. Helpfulness and correctness are strongly correlated, suggesting these criteria capture overlapping aspects of response quality.

7.2 Response Length

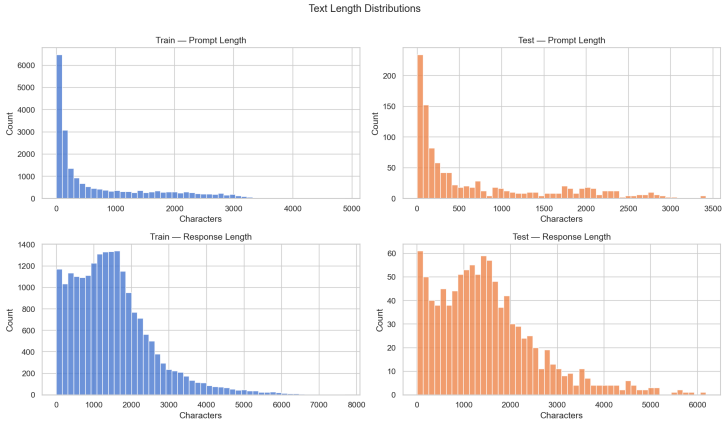

Figure 5: Distribution of prompt and response lengths (in characters).

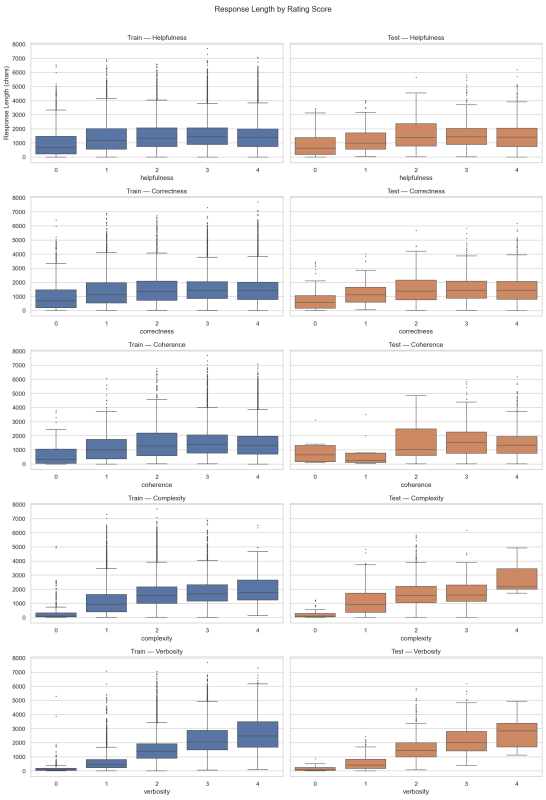

Figure 6: Response length by helpfulness rating. Longer responses tend to receive higher scores.

7.3 Metric Gaps Between Paired Responses

Figure 7: Distribution of per-metric gaps (score of A minus score of B) across all pairs. Larger gaps correspond to easier judgments.

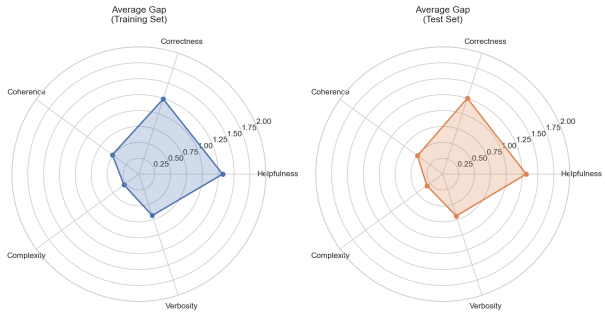

Figure 8: Average metric gaps.

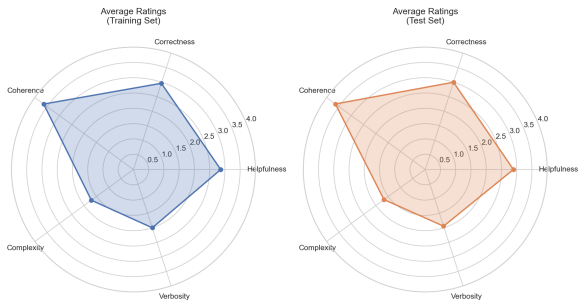

Figure 9: Average ratings.

7.4 Pair-Level Statistics

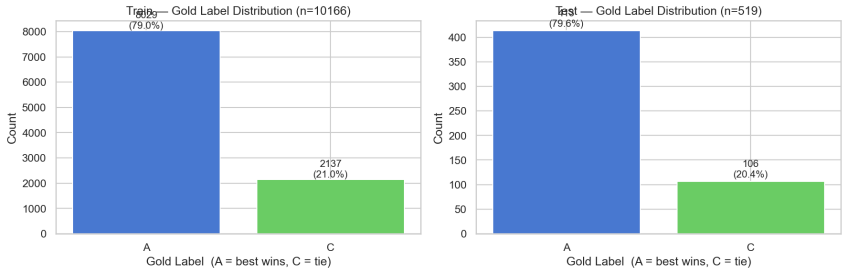

Figure 10: Gold label distribution. The better response is always placed as A, so the majority of labels are A.

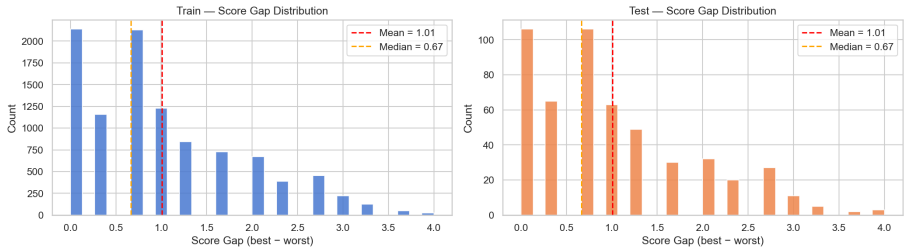

Figure 11: Composite score gap distribution. Smaller gaps indicate pairs that are harder for a judge to distinguish.

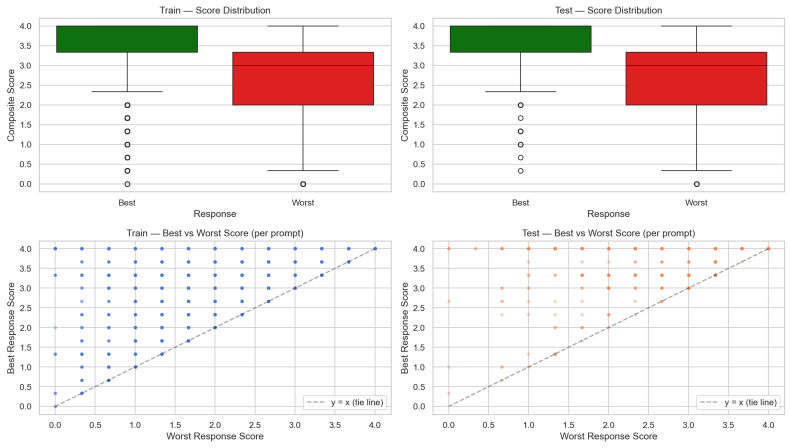

Figure 12: Scatter plot of composite scores: response A versus response B.

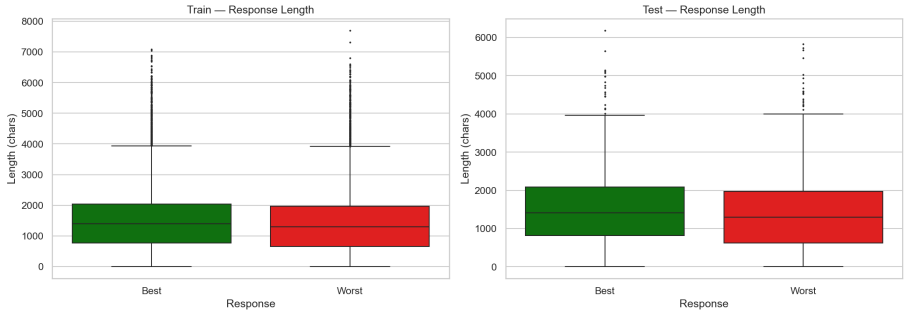

Figure 13: Response length comparison between the better and worse response in each pair.

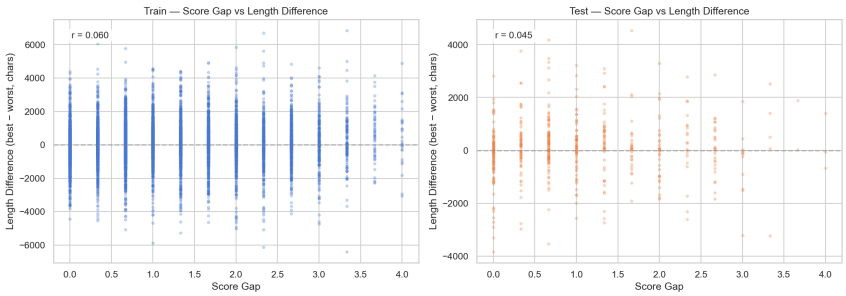

Figure 14: Score gap versus response length difference. Explores whether quality differences correlate with verbosity differences.

7.5 Difficulty buckets

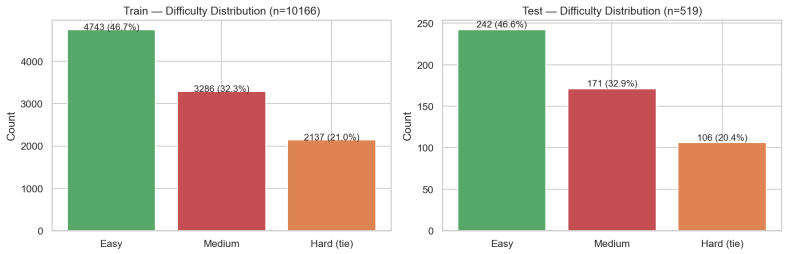

Figure 15: Number of pairs per difficulty bucket. Easy: gap > 1.0; medium: $0.33 <$ gap ≤ 1.0; hard: gap ≤ 0.33.

End of document.